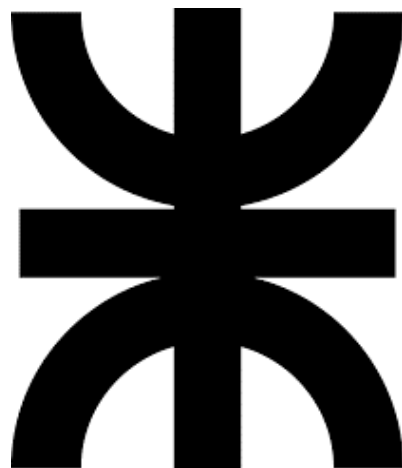


UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA

INGENIERÍA ELECTRÓNICA



TECNICAS DIGITALES II

Trabajo Práctico N°1

PROGRAMACIÓN DE LA PLACA BLUEPILL USANDO GNU ARM
TOOLCHAIN

DOCENTE : Dr. Ing. Steiner Guillermo
Ing. Estefanía Pereyra.

CURSO : 4R2 - Grupo: 13

ALUMNOS	:	Condori, Mariano	406455
		Pera, Sebastián	80456
		Suárez, Rodolfo Eduardo	79669

CÓRDOBA, ARGENTINA
2026

CONTENIDO

1. Introducción	3
2. Actividad Práctica	4
2.1. Archivo main.c	4
2.2. Archivo crt.s	5
2.3. Archivo linker.ld	5
2.3.1. Memoria Flash	5
2.3.2. Memoria RAM	6
2.4. Archivo Makefile	6
2.5. Implementación de un segundo LED y Systick y interrupciones	6
3. Codigos	9

1. Introducción

El objetivo de este trabajo práctico fue desarrollar una aplicación bare metal sobre la placa STM32 BluePill utilizando GNU ARM Toolchain, OpenOCD y programación directa sobre registros del microcontrolador STM32F103. En este trabajo se programó el hardware directamente mediante acceso a registros de memoria.

Durante el desarrollo se trabajó con distintos conceptos fundamentales de sistemas embebidos, como la configuración de GPIO, el manejo del clock de periféricos, la utilización de temporizadores internos del núcleo ARM Cortex-M3 y la programación mediante interrupciones.

Inicialmente se implementó un programa básico capaz de controlar el LED integrado de la placa BluePill utilizando retardos por software mediante bucles vacíos. Posteriormente se realizaron las modificaciones solicitadas durante las clases de laboratorio:

Se agregó un segundo LED conectado a otro pin GPIO. Se reemplazó el retardo basado en bucles por un sistema temporizado utilizando interrupciones del SysTick.

El trabajo permitió comprender con mayor profundidad el funcionamiento interno de un microcontrolador, la organización de memoria y la interacción directa entre software y hardware.

Para el desarrollo del proyecto se utilizó Ubuntu Linux como sistema operativo principal. Se instalaron las herramientas necesarias para compilar, enlazar y grabar el firmware sobre el microcontrolador STM32F103.

Las herramientas utilizadas fueron:

- GNU ARM Toolchain (
textttarm-none-eabi-gcc)
- OpenOCD
- Make
- GDB

2. Actividad Práctica

El proyecto base se encontraba compuesto principalmente por tres archivos fundamentales:

- `main.c`
- `crt.s`
- `linker.ld`
- `Makefile`

Cada uno de estos archivos cumple una función distinta dentro del sistema embebido.

2.1. Archivo `main.c`

El archivo `main.c` contiene la lógica principal de la aplicación. En este archivo se realiza la configuración de los periféricos y se implementa el comportamiento deseado del programa.

En el proyecto base, el objetivo principal era hacer parpadear el LED integrado de la placa BluePill conectado al pin PC13.

Para configurar el LED integrado de la BluePill fue necesario primero habilitar el puerto GPIOC. Esto se realiza activando el clock del periférico mediante el registro `RCC_APB2ENR`. Si no se habilita el clock, el puerto no funciona aunque se intenten modificar sus registros.

Luego se configuró el pin PC13, que es el pin donde está conectado el LED integrado de la placa. Como el pin 13 pertenece al rango de pines 8–15, se utiliza el registro `GPIOC_CRH` para su configuración.

Cada pin GPIO utiliza 4 bits dentro de este registro:

- 2 bits correspondientes al modo (MODE)
- 2 bits correspondientes a la configuración (CNF)

En este caso se configuró el pin como salida digital push-pull con velocidad de 2 MHz. Primero se limpiaron los bits correspondientes al pin 13:

```
GPIOC_CRH &= 0xFF0FFFFFF;
```

Luego se escribieron los bits necesarios para dejar el pin configurado como salida:

```
GPIOC_CRH |= 0x00200000;
```

Una vez configurado el pin, el LED se controla mediante el registro `GPIOC_ODR`, que permite modificar el estado lógico de las salidas del puerto.

Para apagar el LED se coloca el pin en alto:

```
GPIOC_ODR |= GPIOC13;
```

Y para encenderlo se coloca el pin en bajo:

```
GPIOC_ODR &= ~GPIOC13;
```

Es importante tener en cuenta que el LED de la BluePill trabaja con lógica invertida, por lo que:

- Nivel bajo (0) → LED encendido
- Nivel alto (1) → LED apagado

Inicialmente el parpadeo se implementó mediante bucles vacíos:

```
for (int i = 0; i < 1000000; i++);
```

2.2. Archivo crt.s

El archivo `crt.s` implementa el startup del sistema. Este archivo está escrito en lenguaje Assembly y es el primer código ejecutado por el microcontrolador luego de un reset.

Cuando el STM32 se inicia, la CPU Cortex-M3 busca automáticamente la tabla de vectores ubicada al inicio de la memoria Flash.

Dentro de esta tabla se encuentran:

- El valor inicial del Stack Pointer.
- La dirección de la rutina de reset.

2.3. Archivo linker.ld

El archivo `linker.ld` define el mapa de memoria utilizado por el linker.

En este archivo se especifican:

- Las regiones de memoria Flash y RAM.
- Las direcciones de inicio.
- El tamaño de cada memoria.
- La ubicación del stack.

2.3.1. Memoria Flash

```
FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 128K
```

Esta línea indica que:

- La memoria Flash comienza en la dirección 0x08000000.
- Posee un tamaño de 128 KB.
- Tiene permisos de lectura y ejecución.

La memoria Flash almacena el firmware del microcontrolador.

2.3.2. Memoria RAM

RAM (xrw) : ORIGIN = 0x20000000, LENGTH = 20K

Esta región representa la memoria RAM interna del STM32F103

2.4. Archivo Makefile

El archivo Makefile automatiza la compilación y el grabado del proyecto utilizando la herramienta make. En este caso tambien se contienen en este archivo las lineas de correccion del ID de la placa BluePill, ya que en este caso no coincide con el ID de una placa original.

2.5. Implementación de un segundo LED y SysTick y interrupciones

Como primera modificación solicitada, se agregó un segundo LED conectado al pin PB7 del puerto GPIOB en la versión del programa el retardo se realizaba utilizando un bucle vacío:

```
for(int i = 0; i <1000000; i++);
```

Este método funciona, pero no es preciso porque depende de la velocidad del procesador y del compilador. Por eso se reemplazó por un temporizador de hardware llamado SysTick.

El SysTick es un temporizador interno que viene incluido dentro del núcleo Cortex-M3 del STM32. Este temporizador puede contar tiempo automáticamente y generar interrupciones periódicas.

Primero se definieron las direcciones de memoria de los registros del SysTick:

```
#define SysTick_BASE 0xE000E010
#define SysTick_CTRL *(volatile uint32_t *) (SysTick_BASE + 0x00)
#define SysTick_LOAD *(volatile uint32_t *) (SysTick_BASE + 0x04)
#define SysTick_VAL *(volatile uint32_t *) (SysTick_BASE + 0x08)
```

Luego se definieron algunos bits importantes del registro de control:

```
#define SysTick_CTRL_ENABLE (1 << 0)
#define SysTick_CTRL_TICKINT (1 << 1)
#define SysTick_CTRL_CLKSOURCE (1 << 2)
```

También se creó una variable global llamada tick:

```
volatile uint32_t tick;
```

Esta variable se utiliza para guardar la cantidad de milisegundos transcurridos. La interrupción del SysTick se implementó mediante la siguiente función:

```
void SysTick_Handler(void)
{
    tick++;
}
```

Cada vez que el temporizador cumple el tiempo configurado, el hardware ejecuta automáticamente esta función y aumenta la variable `tick` en uno.

Después se configuró el SysTick con la siguiente función:

```
void systick_init_ms(void)
{
    tick = 0;

    SysTick_CTRL &= ~SysTick_CTRL_CLKSOURCE;

    SysTick_LOAD = 999;

    SysTick_VAL = 0;

    SysTick_CTRL |= SysTick_CTRL_TICKINT |
                    SysTick_CTRL_ENABLE;
}
```

El temporizador fue configurado para generar una interrupción cada 1 ms. De esta manera, la variable `tick` aumenta automáticamente cada milisegundo.

Finalmente se implementó la función de delay:

```
void delay_ms_bloqueante(uint32_t ms)
{
    uint32_t tiempo_inicio = tick + ms;

    while (tick != tiempo_inicio);
}
```

Esta función guarda el valor actual de `tick` más la cantidad de milisegundos que se desea esperar. Luego queda esperando hasta que la variable llegue a ese valor.

Por ejemplo, si `tick` vale 1000 y se llama:

```
delay_ms_bloqueante(500);
```

la función esperará hasta que `tick` llegue a 1500.

Con este método el retardo ya no depende de un bucle vacío sino de un temporizador real del hardware, logrando una temporización más precisa.

```
#include <stdint.h>
```

```
#define RCC_BASE 0x40021000
```

```
#define GPIOB_BASE 0x40010C00
```

```
#define RCC_APB2ENR (*(volatile uint32_t *) (RCC_BASE + 0x18))
```

```
#define GPIOB_CRL (*(volatile uint32_t *) (GPIOB_BASE + 0x00))
#define GPIOB_ODR (*(volatile uint32_t *) (GPIOB_BASE + 0x0C))

#define RCC_IOPBEN (1 << 3)
#define GPIOB7 (1UL << 7)

void delay(void)
{
    for(int i = 0; i < 500000; i++);
}

void main(void)
{
    RCC_APB2ENR |= RCC_IOPBEN;

    GPIOB_CRL &= 0xFFFFFFFF;
    GPIOB_CRL |= 0x20000000;

    while(1)
    {
        GPIOB_ODR |= GPIOB7;
        delay();

        GPIOB_ODR &= ~GPIOB7;
        delay();
    }
}
```


3. Codigos

A continuacion se presentan los archivos de codigo finales usados para la implementacion de SysTick con un diodo LED externo parpadeando en paralelo y a destiempo al LED interno de la BluePill

main.c

```
#include <stdint.h>

// Direcciones base
#define RCC_BASE      0x40021000
#define GPIOB_BASE    0x40010C00
#define GPIOC_BASE    0x40011000

// Registros de Reloj y GPIO
#define RCC_APB2ENR   *(volatile uint32_t *)(RCC_BASE + 0x18)
#define GPIOB_CRH     *(volatile uint32_t *)(GPIOB_BASE + 0x04)
#define GPIOC_CRH     *(volatile uint32_t *)(GPIOC_BASE + 0x04)
#define GPIOB_ODR     *(volatile uint32_t *)(GPIOB_BASE + 0x0C)
#define GPIOC_ODR     *(volatile uint32_t *)(GPIOC_BASE + 0x0C)

// Mscaras de bits
#define RCC_IOPBEN    (1 << 3) // Habilitar reloj Puerto B
#define RCC_IOPCEN    (1 << 4) // Habilitar reloj Puerto C
#define GPIOB13       (1 << 13) // Pin 13
#define GPIOC13       (1UL << 13) // Pin 13

// --- Registros SysTick ---
#define SysTick_BASE  0xE000E010
#define SysTick_CTRL  *(volatile uint32_t *)(SysTick_BASE + 0x00)
#define SysTick_LOAD  *(volatile uint32_t *)(SysTick_BASE + 0x04)
#define SysTick_VAL    *(volatile uint32_t *)(SysTick_BASE + 0x08)

volatile uint32_t tick;

void SysTick_Handler(void) { tick++; }

void systick_init_ms(void) {
    tick = 0;
    SysTick_CTRL &= ~(1 << 2); // Reloj a HCLK/8
    SysTick_LOAD = 999;
    SysTick_VAL = 0;
    SysTick_CTRL |= 0x03; // Enable e interrupci n
}
```

```
void main(void) {  
    // 1. Habilitar relojes para Puerto B y C  
    RCC_APB2ENR |= (RCC_IOPBEN | RCC_IOPCEN);  
  
    // 2. Configurar PC13 como salida (LED de la placa)  
    GPIOC_CRH &= 0xFF0FFFFFFF;  
    GPIOC_CRH |= 0x00200000;  
  
    // 3. Configurar PB13 como salida (LED externo)  
    GPIOB_CRH &= 0xFF0FFFFFFF; // Limpiar bits del pin 13  
    GPIOB_CRH |= 0x00200000; // Modo salida 2MHz push-pull  
  
    systick_init_ms();  
  
    while (1) {  
        // --- Lógica del LED de la placa (PC13) ---  
        // Cambia de estado cada 1000ms  
        if ((tick % 1000) < 500) {  
            GPIOC_ODR &= ~GPIOC13; // Encender  
        } else {  
            GPIOC_ODR |= GPIOC13; // Apagar  
        }  
  
        // --- Lógica del LED externo (PB13) ---  
        // Cambia de estado cada 200ms  
        if ((tick % 200) < 100) {  
            GPIOB_ODR |= GPIOB13; // Encender  
        } else {  
            GPIOB_ODR &= ~GPIOB13; // Apagar  
        }  
    }  
}
```

Makefile

```
SRC_DIR = src
BUILD_DIR = build

CC = arm-none-eabi-gcc
AS = arm-none-eabi-as
LD = arm-none-eabi-ld
BIN = arm-none-eabi-objcopy

CFLAGS = -mthumb -mcpu=cortex-m3 -O0 -g

all: app.bin

$(BUILD_DIR):
    mkdir -p $(BUILD_DIR)

crt.o: $(BUILD_DIR) $(SRC_DIR)/crt.s
    $(AS) -o $(BUILD_DIR)/crt.o $(SRC_DIR)/crt.s

main.o: $(BUILD_DIR) $(SRC_DIR)/main.c
    $(CC) $(CFLAGS) -c -o $(BUILD_DIR)/main.o $(SRC_DIR)/main.c

app.elf: crt.o main.o $(SRC_DIR)/linker.ld
    $(LD) -T $(SRC_DIR)/linker.ld -o $(BUILD_DIR)/app.elf \
    $(BUILD_DIR)/crt.o $(BUILD_DIR)/main.o

app.bin: app.elf
    $(BIN) -O binary $(BUILD_DIR)/app.elf $(BUILD_DIR)/app.bin

clean:
    rm -rf $(BUILD_DIR)

flash: app.bin
    openocd -f interface/stlink.cfg \
    -c "set_CPUTAPID_0x2ba01477" \
    -f target/stm32f1x.cfg \
    -c "init" -c "reset_halt" \
    -c "flash_write_image_erase_$(BUILD_DIR)/app.bin_0x08000000" \
    -c "reset_run" -c "exit"

erase:
    openocd -f interface/stlink.cfg \
    -c "set_CPUTAPID_0x2ba01477" \
    -f target/stm32f1x.cfg \
    -c "init" -c "reset_halt" \
    -c "flash_erase_sector_0_0_last" \
    -c "reset_run" -c "exit"
```

crt.s

```
.syntax unified
.cpu cortex-m3
.thumb

.global _esstack
.global _reset

.word _esstack
.word _reset

.thumb_func
_reset:
    bl main
    b .
```

linker.ld

```
MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000 , LENGTH = 128K
    RAM (xrw)  : ORIGIN = 0x20000000 , LENGTH = 20K
}

_esstack = ORIGIN(RAM) + LENGTH(RAM);
```